# Smart Quiz App: Components and Viva Guide

Shaurya Singh | 21CSC203P Advanced Programming Practice | Unit 5: Event Programming Paradigm

## 1. Your 30-second introduction

My project is a Smart Quiz App built with Python and Tkinter. The user enters a name, chooses a category (Python, Java Basics or Unit 5 Concepts), the number of questions and the time per question. Each question appears with four option cards and a countdown bar. After every answer the correct option turns green and a wrong pick turns red. At the end the app shows the score as a pie chart and an answer review. The questions are stored in a JSON file, so they can be changed without touching the code. The whole program is event-driven: it waits for events such as clicks, key presses and timer ticks, and runs a handler function for each one.

## 2. Files and program flow

- **quiz_app.py**: the application (about 600 lines including the built-in questions).
- **questions.json**: the question bank, 15 questions in each of 3 categories.
- **README.md** and **.gitignore**: for GitHub.

**Flow:** start, load questions, Home screen, start_quiz, show_question (repeated for every question), show_result, show_review.

**quiz_app.py has three parts:**

1. Question data: DEFAULT_QUESTIONS, parse_question, load_questions, shuffle_options.
2. Look and feel: the THEMES dictionary and constants such as BAR_WIDTH and FEEDBACK_MS.
3. The QuizApp class: every screen and every event handler.

Each screen (home, quiz, result, review) is a **Frame**. To change screens, new_screen() cancels the timer, destroys the old frame and creates a fresh one.

# 3. Components used, one by one

## Tk() and mainloop()

- root = tk.Tk() creates the main window. root.mainloop() starts the **event loop**: an endless loop that waits for events (click, key press, timer) and calls the matching handler. The program stays in it until the window closes.
- Used with: title(), resizable(False, False) to lock the size, and geometry() inside center_window() to place the window in the middle of the screen.
- **Say:** 'mainloop keeps the window alive and listens for events. Without it the window would open and close instantly.'

## Frame

A container that groups widgets. I use one Frame per screen, a 'form' Frame for the grid layout on the home screen, and small Frames for rows of buttons. A separate Frame is needed because **pack and grid must not be mixed in the same parent**.

## Geometry managers: pack, grid, place

| Manager | How it works | Where I used it |
|---|---|---|
| pack() | Stacks widgets in blocks. Options: side, fill, expand, padx, pady | Screens, labels, buttons, option cards |
| grid() | Rows and columns. Options: row, column, sticky, padx, pady | The home screen form (labels in column 0, inputs in column 1) |
| place() | Exact position by x, y or relative relx, rely | The score label fixed at the top-right corner (relx=1.0, x=-15, y=12, anchor='ne') |

## Label

Shows text. Options: text, font, fg (text colour), bg (background), wraplength (wraps long text), justify. I use it for titles, the question number, the score, the timer text and the Correct/Wrong message. The text is changed later with .config(text=...).

## Button

A clickable widget. **command=function_name** is called when it is clicked, written **without brackets** (with brackets the function would run immediately). Options: bg, fg, width, relief, cursor, state. I set state='disabled' on Next during feedback. A helper method btn() creates every button so the styling is not repeated.

## Entry

A single-line text box for the player name. It is linked to name_var with textvariable. The name is checked with strip(), must not be empty and must be at most 20 characters.

## Combobox (from ttk)

A drop-down list for the category. values=list(self.bank) means the categories come from the JSON file automatically. state='readonly' stops the user typing a wrong category. Combobox exists only in **ttk** (themed widgets), which is why it is imported from tkinter.ttk. I use the 'clam' ttk theme so the colours can follow dark mode.

## Spinbox

Up/down number picker for the number of questions (3 to 15). Options: from_, to, textvariable, state='readonly'. It returns a **string**, so I convert it with int() inside a try/except, and the quiz never uses more questions than the category contains.

## Scale

A slider for seconds per question (10 to 60). Options: from_, to, resolution=5 (moves in steps of 5), orient='horizontal', variable=sec_var.

## Checkbutton

An independent ON/OFF toggle that stores 1 or 0 in an IntVar. I have two: 'Shuffle questions and options' and 'Enable timer'.

## Radiobutton

- A group of buttons that share **one variable** (self.choice). Each has its own value (0, 1, 2, 3). Selecting one automatically deselects the others, so only one answer can be chosen.

- self.choice = -1 means nothing is selected, which lets me show a warning if the user presses Next without choosing.
- **indicatoron=0** removes the small circle and draws the Radiobutton as a clickable card. selectcolor is the colour of the selected card.
- During feedback the cards are set to state='disabled' so the answer cannot be changed.
- **Difference from Checkbutton:** radiobuttons are mutually exclusive, checkbuttons are independent.

## Canvas

A drawing area. The origin (0, 0) is the top-left corner and y increases downward.

- **Timer bar:** create_rectangle() draws it once. Every second, coords() shrinks its width to BAR_WIDTH x time_left / time_total, and itemconfig() turns it red when 25 percent of the time is left.
- **Pie chart:** create_arc(start=90, extent=360 x score / total) draws the green slice and a second arc draws the red slice. A 360-degree arc cannot be drawn, so for 100 percent or 0 percent I use create_oval() instead.
- create_text() and small rectangles draw the legend (Correct / Wrong).

## Text and Scrollbar

Used on the review screen. Text.insert('end', text, tag) adds lines. **Tags** (tag_config) give styles such as bold or green/red text. state='disabled' makes it read-only. The Scrollbar is linked both ways: Text(yscrollcommand=scroll.set) and scroll.config(command=text.yview).

## Menu

tk.Menu(root) is the menu bar. Submenus are created with Menu(bar, tearoff=0) (tearoff=0 removes the dashed detach line). Methods: add_command, add_separator, add_cascade (attaches a submenu) and add_checkbutton (the Dark mode tick). root.config(menu=bar) shows it. A Menubutton (in your Unit 5 PPT) is the same kind of drop-down but placed as a normal widget.

## messagebox

Pop-up dialogs: showinfo, showwarning, showerror and askyesno (returns True or False). Used for the empty-name error, 'choose an answer' warning, JSON problems, the quit confirmation and the Help/About boxes.

## Tkinter variables

StringVar, IntVar (also DoubleVar and BooleanVar) are objects linked to widgets. .get() reads the value and .set() changes it, and the widget updates automatically. Used: name_var, category_var, count_var, sec_var, shuffle_var, timer_var, dark_var and choice.

## Events, callbacks and timers

| Tool | What it does | Where it is used |
|---|---|---|
| command= | Calls a function when a widget is clicked | Buttons, menu items, Dark mode toggle |
| bind(sequence, handler) | Calls a handler for a keyboard or mouse event. The handler receives an **event** object | Enter key (on_enter), A-D keys (on_key, uses event.char), mouse enter/leave hover on option cards |
| after(ms, function) | Runs a function once after a delay without freezing the window | Timer tick every 1000 ms, 1-second feedback delay, start-up warning |
| after_cancel(id) | Cancels a scheduled call | cancel_timer() |
| protocol('WM_DELETE_WINDOW', function) | Runs a function when the X button is pressed | quit_app (asks for confirmation during a quiz) |

**Why after() and not a while loop with sleep()?** sleep() blocks the event loop, so the window would freeze and nothing could be clicked. after() lets the event loop keep running.

**Why lambda e, w=rb in the hover bindings?** Inside a loop, plain lambda would use the last widget. The default argument w=rb captures the current widget for each lambda.

### Themes

THEMES is a dictionary of colours for light and dark. apply_theme() sets the window and ttk colours. toggle_theme() redraws the current screen and keeps your selected option and remaining time.

## 4. Python concepts used

| Concept | Where | Why |
|---|---|---|
| Class and self | QuizApp | Keeps state (score, index, answers) as attributes shared by all methods |
| dict, list, tuple | bank = {category: [(text, options, answer_index)]} | Tuples hold one fixed question record |
| json module | load_questions | json.load() converts JSON text into Python dict/list |
| with open(...) | load_questions | Closes the file automatically, even on errors |
| try / except | JSON loading, int() conversion | Handles FileNotFoundError, JSONDecodeError, OSError, UnicodeDecodeError, ValueError |
| raise ValueError | parse_question | Reports exactly why a question is invalid |
| pathlib.Path | Path(**file**).with_name('questions.json') | Finds the file next to the script from any folder |
| random.shuffle | start_quiz, shuffle_options | Random question and option order |
| list slicing, min, max | pool[:count] | Never asks for more questions than exist |
| f-strings | Labels and messages | Easy text formatting |
| constants | THEMES, BAR_WIDTH, FEEDBACK_MS | No magic numbers; one place to change |

| Concept | Where | Why |
|---|---|---|
| helper methods | lbl(), btn(), check() | Avoids repeated code (DRY) |
| **name** check | last lines | Runs the app only when the file is executed directly |

## 5. Function walk-through

- **parse_question(item):** checks one JSON question (text present, exactly 4 different options, answer matches an option) and returns (text, options, answer_index), or raises ValueError.
- **load_questions(path):** reads questions.json. If the file is missing or broken it returns the built-in DEFAULT_QUESTIONS. Bad questions are skipped and reported in a list of notes.
- **shuffle_options(q):** copies the options, shuffles them and finds the new index of the correct text.
- **QuizApp.init:** creates the Tkinter variables, centres the window, applies the theme, builds the menu and Home screen, and binds the keyboard events.
- **new_screen():** cancels the timer, unlocks, destroys the old frame and returns a new one.
- **show_home():** the settings form (Entry, Combobox, Spinbox, Scale, Checkbuttons).
- **start_quiz():** validates the name and category, picks and shuffles the questions, fixes the timer settings and shows the first question.
- **show_question():** builds the question card, the option Radiobuttons and the timer bar, and starts the timer last, after all widgets exist.
- **tick():** redraws the bar every second. At 0 seconds it calls next_question(timed_out=True).
- **next_question():** ignored if locked; warns if nothing is selected; stores the answer, updates the score, sets locked=True, shows feedback and schedules advance() after 1 second.
- **show_feedback():** correct option green, wrong pick red, options disabled, message shown.
- **advance():** moves to the next question or to the result screen.
- **show_result():** score text, remark, pie chart and the three buttons.
- **show_review():** every question with your answer and the correct answer.

- **on_enter / on_key:** keyboard handlers (Enter for Next, A-D or 1-4 to choose).
- **quit_app():** asks for confirmation only while a quiz is running.
- **reload_questions():** re-reads questions.json without restarting.

## 6. Viva questions and answers

### A. About the project

- **Why did you choose a quiz app?** It uses many widgets and events, it is useful for students, and the logic is simple enough to explain fully.
- **What makes it different?** Instant green/red feedback, answer review, dark mode, shuffled options, a Unit 5 category, and questions in a validated JSON file with live reload.
- **Which programming paradigm does it follow?** Event-driven programming: the flow is decided by user events, not by a fixed sequence.
- **Which libraries did you use?** tkinter (with ttk and messagebox), json, random and pathlib. All are in the standard library, so nothing needs installing.
- **Does it run on other systems?** Yes. On Linux install python3-tk. On macOS the button background colours may be ignored by the system.
- **What problems did you face?** Use the ones that are true for you, for example: the timer must not freeze the window (solved with after), the answer index had to be updated after shuffling options, and widgets must exist before the timer starts.

### B. Tkinter

- **What is Tkinter?** The standard GUI library of Python. It is a Python interface to the Tk toolkit.
- **Name other Python GUI libraries.** wxPython, PyQt and JPython (from your PPT), and Kivy.
- **Steps to create a Tkinter app?** Import tkinter, create the main window with Tk(), add widgets, then call mainloop().
- **What is a widget?** A GUI control such as a Button, Label or Entry.
- **Difference between pack, grid and place?** pack stacks in blocks, grid uses rows and columns, place uses exact coordinates. Do not mix pack and grid in the same parent.
- **Checkbutton vs Radiobutton?** Checkbutton is an independent on/off toggle. Radiobuttons share one variable and allow only one choice.

- **Why use StringVar/IntVar instead of reading the widget?** The variable stays linked to the widget, so changes in either place update the other, and several widgets can share one variable (the Radiobuttons).
- **Why command=func and not command=func()?** With brackets the function runs immediately when the button is created. Without brackets Tkinter stores it and calls it on click.
- **command vs bind?** command is built into a widget's main action. bind attaches a handler to any event (key, mouse move) and passes an event object.
- **What is an event loop?** The loop inside mainloop() that waits for events and dispatches them to handlers.
- **What does after() do?** Schedules a function to run once after some milliseconds without blocking the GUI. My timer re-schedules itself every 1000 ms.
- **Why destroy the frame instead of hiding it?** It frees the old widgets, keeps the code simple, and lets the new screen use the current theme.
- **What does resizable(False, False) do?** Stops the user from resizing the window in width and height.
- **What is ttk?** Themed Tk widgets. I use ttk for the Combobox and ttk.Style to colour it.
- **How is the timer bar drawn?** One rectangle on a Canvas. Each second its width is set to BAR_WIDTH x time_left / time_total using coords().
- **How is the pie chart drawn?** Two create_arc calls from 90 degrees. The green extent is 360 x score / total and the red arc covers the rest.
- **What are tags in the Text widget?** Named styles (bold, colour) applied to inserted text.
- **What does state='disabled' do?** The widget no longer reacts to the user. I use it to lock options during feedback and to make the review Text read-only.
- **What do sticky and anchor mean?** sticky (grid) says which sides of the cell the widget sticks to. anchor says where text or a widget sits inside its space, for example 'w' for left.

## C. Python and code

- **Why a JSON file for questions?** It separates data from logic. A teacher can update the quiz without knowing Python.
- **json.load vs json.loads vs json.dump?** load reads from a file, loads reads from a string, dump writes to a file.

- **What does with open() do?** Opens the file and closes it automatically, even if an error happens.
- **Which exceptions do you handle?** FileNotFoundError (missing file), JSONDecodeError (bad syntax), OSError and UnicodeDecodeError (cannot read), ValueError (invalid question or number).
- **Why does the JSON store the answer text, not an index?** It is easier and safer for a person to edit. The code converts it to an index.
- **Why options[:] before shuffling?** It makes a copy so the original list is not changed.
- **What is self?** The current object. It lets methods share data like self.score and self.index.
- **What is a lambda?** A small anonymous function, used here for the hover bindings and menu help boxes.
- **Why tuples for questions?** A question is a fixed record that should not change.
- **What does if name == 'main' do?** Runs the app only when the file is run directly, not when it is imported.
- **What does Path(file).with_name() do?** Builds the path of questions.json in the same folder as the script, so it works from any working directory.

## D. Design and what-if

- **What if questions.json is missing or broken?** The app falls back to the built-in questions and shows a warning if the file was broken.
- **What if one question is invalid?** Only that question is skipped and the warning says which one and why.
- **What if the user clicks Next twice quickly?** The locked flag ignores the second click while feedback is showing.
- **What if the category has fewer questions than selected?** The quiz uses min(count, available).
- **What if the user closes the window during the timer?** protocol('WM_DELETE_WINDOW') calls quit_app, which asks for confirmation, cancels the timer and destroys the window.
- **What are the limitations?** Single player, scores are not saved, exactly 4 options per question, fixed window size.
- **Future scope?** Leaderboard saved to a file, an Add Question window, a 50-50 lifeline, sound, online questions or a database.

- **How did you test it?** Manual play-through, plus automated checks of the JSON loader, scoring, timer timeout, shuffling and theme switching.

### E. Unit 5 theory (from your PPT)

- **What is event-driven programming?** A paradigm where program flow is decided by events such as a mouse click, key press or message. The main routine is an event loop that waits for an event and calls the right handler.
- **Event handler?** A function that runs a specific action when a specific event occurs.
- **Trigger function?** A function that decides which handler to run when an event occurs.
- **Event callback?** A function invoked when something significant happens, such as a click.
- **What is a DFA?** A 5-tuple (Q, Sigma, delta, q0, F): states, alphabet, transition function, start state and accepting states. Every state has exactly one transition for each symbol.
- **DFA vs NFA?** In an NFA the transition goes to a set of states, and it can have several transitions for one symbol and epsilon moves.
- **Which library builds DFA/NFA in Python?** automata-lib (from automata.fa.dfa import DFA).
- **What is symbolic programming?** A paradigm where a program manipulates formulas as data. In Python the library is **SymPy** (pip install sympy). It supports symbols(), expand(), simplify(), diff(), integrate(), limit(), series() and solve().
- **How does your project connect to Unit 5?** The quiz app is a Tkinter program, and Tkinter is the event programming paradigm: widgets, events, handlers and the event loop.

## 7. Changes they may ask you to make live

| Request | Where | What to change |
|---|---|---|
| Change default seconds | **init** | self.sec_var = tk.IntVar(value=20) |
| Change feedback delay | constants | FEEDBACK_MS = 1000 |
| Change colours | constants | The THEMES dictionary |
| Change window size | constants | WIN_W, WIN_H |

| Request | Where | What to change |
|---|---|---|
| Change timer bar length | constants | BAR_WIDTH |
| Change pass mark or remark | show_result | The percent >= 80 and percent >= 50 conditions |
| Add a question | questions.json | Add an object, then File > Reload Questions |
| Add a category | questions.json | Add a new top-level name with at least one valid question |
| Change maximum name length | constants | MAX_NAME = 20 |
| Negative marking | next_question | Add else: self.score -= 0.25 after the correct check (the score becomes a decimal) |

## 8. Quick revision

| Widget or tool | One-line answer |
|---|---|
| Tk / mainloop | Main window and the event loop |
| Frame | Container; one per screen |
| Label | Shows text |
| Button | Runs a function on click (command) |
| Entry | One-line text input |
| Combobox | Drop-down list (ttk) |
| Spinbox | Number picker |
| Scale | Slider |
| Checkbutton | Independent on/off |
| Radiobutton | One choice from many; shared variable |
| Canvas | Drawing: timer bar and pie chart |
| Text + Scrollbar | Scrollable styled text for the review |

| Widget or tool | One-line answer |
|---|---|
| Menu | Menu bar with File, View, Help |
| messagebox | Pop-up info, warning, error, yes/no |
| bind | Attach a handler to key or mouse events |
| after | Run a function later without freezing the GUI |
| pack / grid / place | Block, table and exact-position layouts |